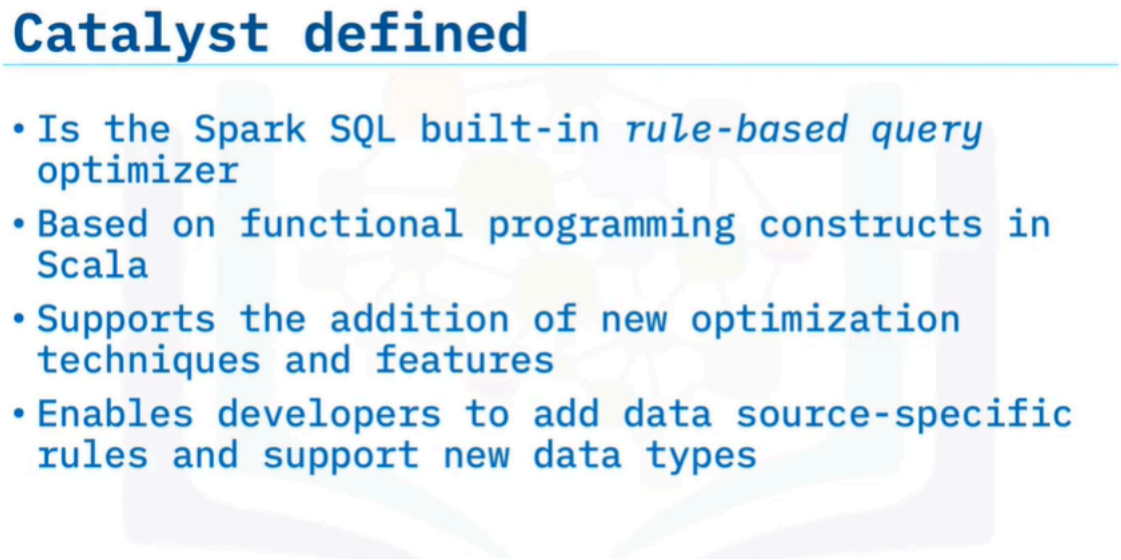
Catalyst and Tungsten

Sure! Let's talk about the concept of **Catalyst and Tungsten in Spark SQL Optimization**.

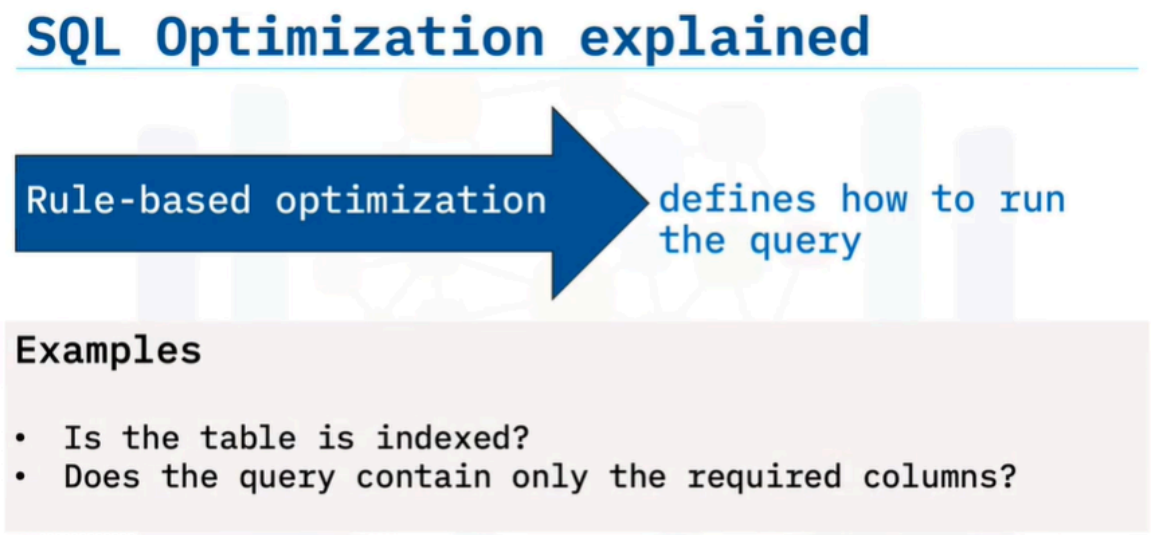
Catalyst is like a smart assistant for Spark SQL that helps make your SQL queries run faster and use less memory. Imagine you have a recipe for a delicious cake, but it has some unnecessary steps that slow you down. Catalyst looks at your recipe (the SQL query) and removes those extra steps, making it quicker to bake your cake. It does this by following specific rules, like checking if you have the right ingredients (data) and only using what you need.

On the other hand, Tungsten is like a mechanic who fine-tunes your car for better performance. While Catalyst optimizes the recipe, Tungsten focuses on how the engine (the computer) works. It makes sure that the car runs smoothly by using the best fuel (memory and CPU) and avoiding any unnecessary stops (like garbage collection). This way, your car can go faster and use less gas, just like how Tungsten helps Spark SQL run efficiently.

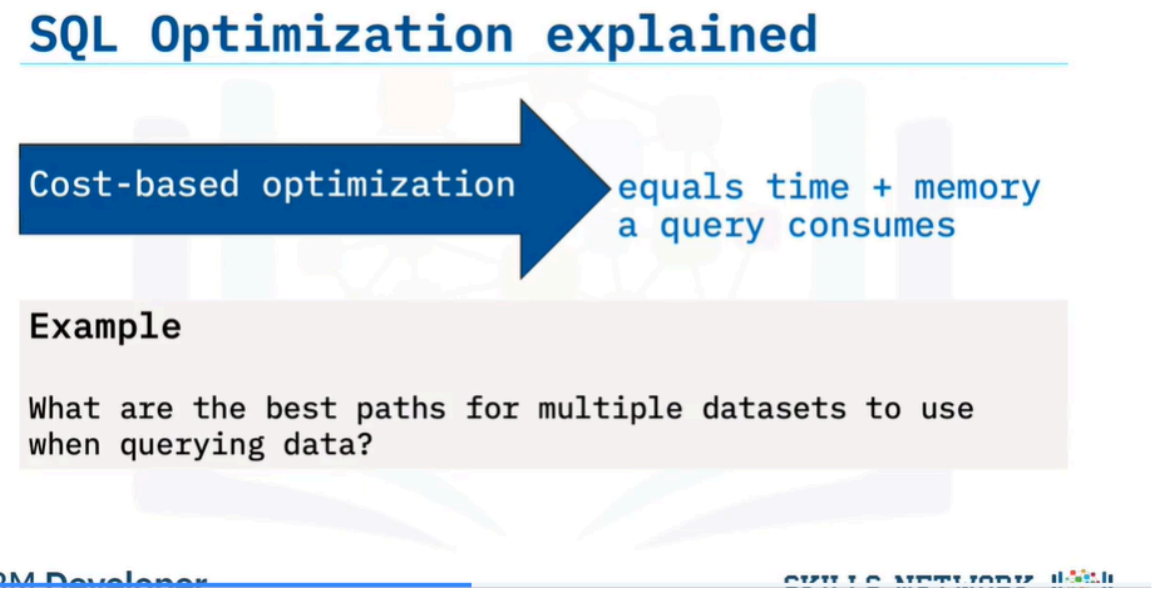
- The primary goal of Spark SQL Optimization is to improve the SQL query run-time performance reducing the query's time and memory consumption, saving organizations time and money.



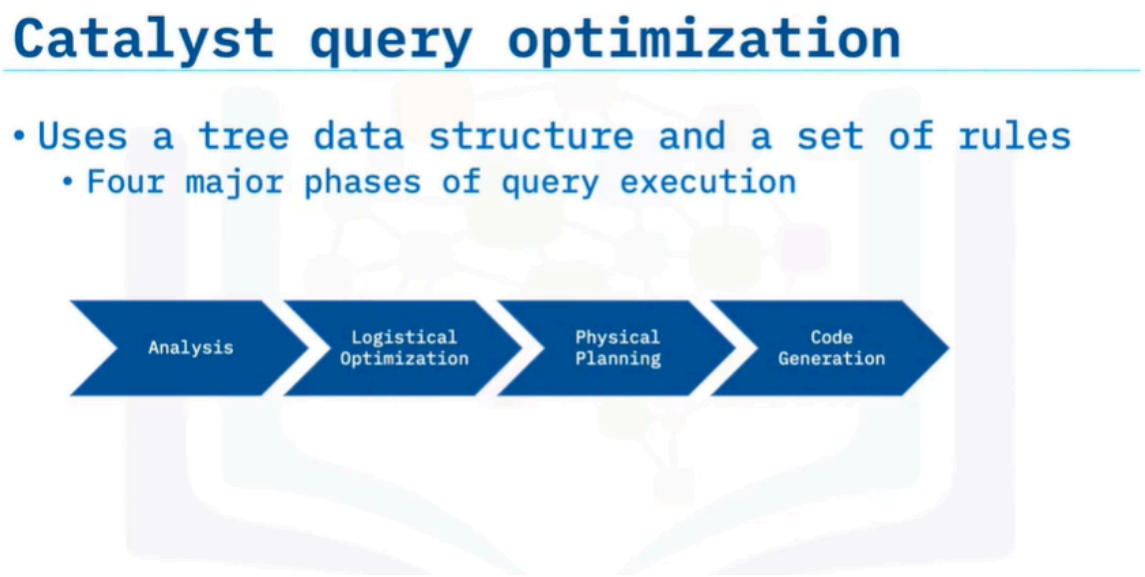
- Spark SQL supports both rule-based and cost-based query optimization. Catalyst, also known as the Catalyst Optimizer, is the Spark SQL built-in rule-based query optimizer. Based on Scala functional programming constructs, Catalyst is designed to easily add new optimization techniques and features to Spark SQL. Developers can extend the optimizer by adding data-source-specific rules and support for new data types.



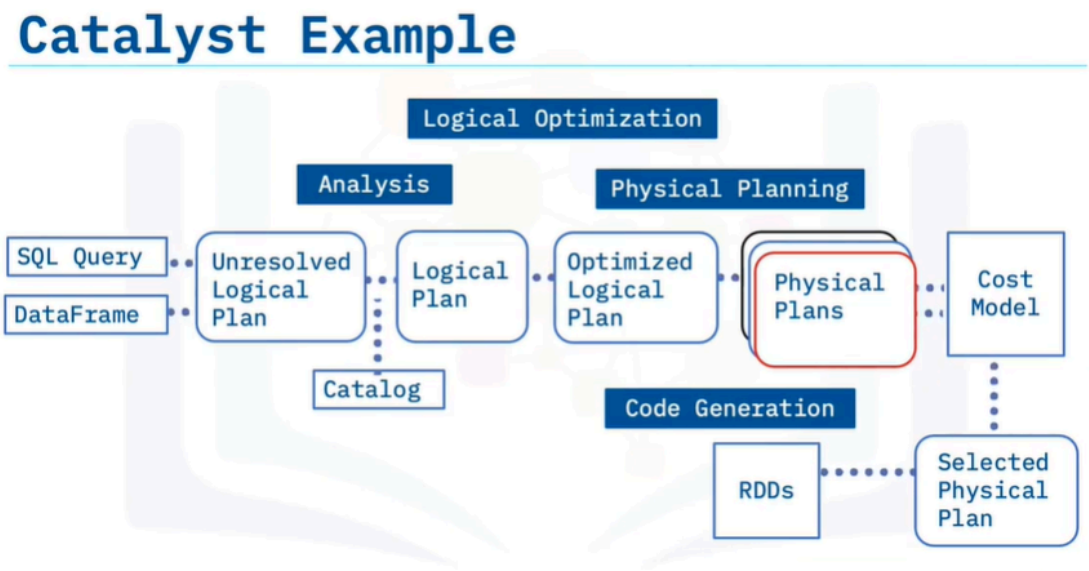
- During rule-based optimization, the SQL optimizer follows predefined rules that determine how to run the SQL query. Examples of predefined rules include validating that a table is indexed and checking that a query contains only the required columns.



- With the query itself optimized, cost-based optimization measures and calculates cost based on the time and memory that the query consumes. The Catalyst optimizer selects the query path that results in the lowest time and memory consumption. Because queries can use multiple paths, these calculations can become quite complex when large datasets are part of the calculation. Consider the analogy of a car and your travels. A car owner can optimize the vehicle's performance with the right oil, tires, fuel, and so on. These actions are similar to rule-based optimization. When it's time to plan a travel route that can save both time and wear and tear on your car, those actions are a form of cost-based optimization.



- In the background, the Catalyst Optimizer uses a tree data structure and provides the data tree rule sets. To optimize a query, Catalyst performs the following four high-level tasks or phases: analysis, logical optimization, physical planning, and code generation.



- Let’s review the four Catalyst phases: In the Analysis phase, Catalyst analyzes the query, the DataFrame, the unresolved logical plan, and the Catalog to create a logical plan. During the Logical Optimization phase, the Logical plan evolves into an Optimized Logical Plan. This is the rule-based optimization step of Spark SQL and rules such as folding, pushdown, and pruning are applied here. During the Physical Planning phase, Catalyst generates multiple physical plans based on the Logical Plan.
- A physical plan describes computation on datasets with specific definitions on how to conduct the computation. A cost model then chooses the physical plan with the least cost. This is the cost-based optimization step. The final phase is Code Generation. Catalyst applies the selected physical plan and generates Java bytecode to run on each node.

Tungsten defined

Spark's *cost-based* optimizer that maximizes *CPU and memory* performance

- Let's now examine how Tungsten provides cost-based optimization in Spark. Tungsten optimizes the performance of underlying hardware focusing on CPU performance instead of IO. Java was initially designed for transactional applications. Tungsten aims to improve CPU and Memory performance by using a method more suited to data processing for the JVM to process data.

Tungsten Features

- Manages memory explicitly and does not rely on the JVM object model or garbage collection
- Enables cache-friendly computation of algorithms and data structures using both STRIDE-based memory access
- Supports on-demand JVM byte code generation
- Does not generate virtual function dispatches
- Places intermediate data in CPU registers
- Enables Loop unrolling

- To achieve optimal CPU performance, Tungsten applies the following capabilities: Tungsten manages memory explicitly and does not rely on the JVM object model or garbage collection. Tungsten creates cache-friendly data structures that are arranged easily and more securely using STRIDE-based memory access instead of random memory access. Tungsten supports JVM bytecode on-demand. Tungsten does not enable virtual function dispatches, reducing multiple CPU calls. Tungsten places intermediate data in CPU registers and enables loop unrolling.